

Practical_05

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

OUTPUT's:-

```
v_santosh@DESKTOP-E34M7RI:~$ mkdir Practical_05
mkdir: Practical_05: File exists
v_santosh@DESKTOP-E34M7RI:~$ cd Practical_05
v_santosh@DESKTOP-E34M7RI:~/Practical_05$
v_santosh@DESKTOP-E34M7RI:~/Practical_05$ nano Practical_05.c
v_santosh@DESKTOP-E34M7RI:~/Practical_05$ gcc Practical_05.c -o Practical_05
v_santosh@DESKTOP-E34M7RI:~/Practical_05$ ./Practical_05
Consumer received:
Data 1 from Producer
Data 2 from Producer
Data 3 from Producer
Data 4 from Producer
Data 5 from Producer
Data 6 from Producer
Data 7 from Producer
Data 8 from Producer
Data 9 from Producer
Data 10 from Producer

Producer: Data sent successfully.
Communication time: 0.000027 seconds
v_santosh@DESKTOP-E34M7RI:~/Practical_05$ nano Practical_05_2.c
v_santosh@DESKTOP-E34M7RI:~/Practical_05$ gcc Practical_05_2.c -o Practical_05_2
v_santosh@DESKTOP-E34M7RI:~/Practical_05$ ./Practical_05_2
-rwxr-xr-x 1 v_santosh v_santosh 16496 Sep 16 06:44 Practical_05
-rw-r--r-- 1 v_santosh v_santosh 1246 Sep 16 06:44 Practical_05.c
-rwxr-xr-x 1 v_santosh v_santosh 16304 Sep 16 06:46 Practical_05_2
-rw-r--r-- 1 v_santosh v_santosh 1146 Sep 16 06:46 Practical_05_2.c
```

Code for Practical_05:-

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main() {
    int fd[2];
    pid_t pid;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        // Parent = Producer
        close(fd[0]);

        char data[100];

        clock_t start = clock();

        for (int i = 1; i <= 10; i++) {
            sprintf(data, "Data %d from Producer\n", i);
            write(fd[1], data, strlen(data));
        }

        close(fd[1]);

        wait(NULL);

        clock_t end = clock();

        double time_taken =
            (double)(end - start) / CLOCKS_PER_SEC;

        printf("\nProducer: Data sent successfully.\n");
        printf("Communication time: %.6f seconds\n", time_taken);
    }
    else {
        // Child = Consumer
        close(fd[1]);

        char buffer[100];
        int n;

        printf("Consumer received:\n");

        while ((n = read(fd[0], buffer, sizeof(buffer) - 1)) > 0) {
            buffer[n] = '\0';
            printf("%s", buffer);
        }

        close(fd[0]);
        exit(0);
    }

    return 0;
}
```

Code for Practical_05_2:-

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    // Create first child for ls -l
    pid_t pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        // Child 1: ls -l

        close(fd[0]);

        // Send stdout to pipe
        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    // Create second child for grep
    pid_t pid2 = fork();

    if (pid2 < 0) {
        perror("fork");
        return 1;
    }

    if (pid2 == 0) {
        // Child 2: grep ".c"

        close(fd[1]);

        // Take stdin from pipe
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

    // Parent closes both pipe ends
    close(fd[0]);
    close(fd[1]);

    // Wait for both children
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```